Retry Pattern: An Example

Let's illuminate our understanding of the Retry Pattern by illustrating its implementation in Java. With a hands-on approach, we'll walk through each aspect of the Retry Pattern, showcasing how we can infuse resilience into our applications.

Setting Up the Stage

To start off, we'll need a method to simulate a service call that could fail intermittently. We'll use a simple method, `unreliableServiceCall()`, which randomly throws an exception to simulate a failure.

```
public void unreliableServiceCall() throws Exception {
    if (new Random().nextInt(10) < 7) { // 70% chance of failure
        throw new Exception("Service is unavailable");
    }
}
```

Building the Retry Logic

Armed with our faulty service method, let's construct our retry logic. We'll create a method, `executeWithRetry()`, that takes a `Callable<Void>` object representing the operation to retry, a maximum retry count, and a delay between retries.

```
public void executeWithRetry(Callable<Void> operation, int maxRetries, long delay) {
    int attempt = 0;
    while (attempt < maxRetries) {
        try {
            operation.call();
            return;
        } catch (Exception ex) {
            attempt++;
            System.out.println("Attempt " + attempt + " failed with message: " +
ex.getMessage());
            sleep(delay);
        }
    }
    System.out.println("All attempts failed");
}
```

The `executeWithRetry()` method catches any exception from the `operation.call()` and retries the operation after the specified delay until the maximum number of retries is reached.

Testing Our Retry Logic

Now, let's put our retry logic to the test with the `unreliableServiceCall()` method.

```
executeWithRetry(this::unreliableServiceCall, 5, 1000);
```

In this code, we're using a method reference to pass the `unreliableServiceCall()` method as the operation to retry. We're allowing up to 5 retries and setting a 1-second delay between retries.

Upon execution, you'll see that the retry logic attempts to call the unreliable service multiple times until it either succeeds or exhausts all retries.

Using a Retry Library

Although our hand-crafted retry logic serves its purpose, it's relatively simple. Real-world scenarios require more sophisticated mechanisms like retry policies and backoff strategies. Luckily, several libraries in Java offer these features out-of-the-box.

One such library is Resilience4j. Let's modify our `executeWithRetry()` method to use Resilience4j's Retry module.

```
public void executeWithRetryResilience4j(Callable<Void> operation, int maxRetries, long
delay) {
    RetryConfig config = RetryConfig.custom()
            .maxAttempts(maxRetries)
            .waitDuration(Duration.ofMillis(delay))
            .retryOnException(e -> e instanceof Exception)
            .build();

    Retry retry = Retry.of("id", config);
    CheckedRunnable retryableOperation = Retry.decorateCheckedRunnable(retry,
operation::call);

    try {
        retryableOperation.run();
    } catch (Throwable throwable) {
        System.out.println("All attempts failed");
    }
}
```

This version of `executeWithRetry()` provides a flexible and powerful approach to handling retries. We've defined a `RetryConfig` that sets the maximum attempts, the delay, and the retry condition. We then create a `Retry` instance and decorate our operation with it.

A Word of Caution

Looking at the example, you may think, "This is simple. I can use this everywhere." But, as with all patterns, the Retry Pattern is not a one-size-fits-all solution. There are considerations to take into account, like knowing when to give up, or preventing a storm of retries from overwhelming your system or the service you're calling. It's not just about making it work—it's about making it work in a sustainable, responsible manner.

Going Further

This is a basic example and real-world scenarios can get much more complex. We might need to consider varying the delay between retries, or using exponential backoff. There are many open-source libraries like Resilience4J and Spring Retry that provide these advanced features.

However, this basic example serves as a good starting point. It illustrates the core of the Retry Pattern: persistently attempting an operation in the face of temporary failures.

Final Thoughts

Understanding the Retry Pattern at its most fundamental level—how to implement it in simple terms—gives us a strong foundation. From here, we can build more complex, robust systems that can withstand the unpredictability of remote services.

Remember, coding isn't just about writing lines of code—it's about problem-solving, it's about resilience. As you're coding, consider how the Retry Pattern, a tool for resilience, can be used in your systems. After all, who doesn't like a system that can pick itself up after a fall, dust itself off, and keep going?

← Previous
The Inner Workings Of The Retry Pattern

Next →
Performance Implications

☐ Mark as Completed